

Legal RAG Assistant

Project Report

Submitted By

Tanishka Shree S

1. Project Title

Legal Intelligence Assistant – Retrieval Augmented Generation (RAG) System for Legal Document Search and Analysis

2. Project Overview

The Legal RAG Assistant is an AI-powered legal document retrieval and question-answering system developed using Retrieval-Augmented Generation (RAG) architecture.

The application combines vector databases, semantic search, and Large Language Models (LLMs) to provide intelligent answers from legal documents. Users can search legal information, retrieve relevant documents, analyze legal topics, and visualize insights through an interactive Streamlit dashboard.

The system leverages ChromaDB for vector storage, Sentence Transformers for embeddings, LangChain for orchestration, and Ollama (Llama 3.2) for response generation.

3. Problem Statement

Legal professionals often deal with large volumes of legal documents such as contracts, compliance reports, employment laws, and regulatory guidelines.

Traditional keyword-based search systems fail to understand the semantic meaning of user queries and often return irrelevant results.

The objective of this project is to build an intelligent legal assistant capable of:

- Understanding natural language queries
- Retrieving relevant legal information
- Generating context-aware responses
- Providing analytical insights through dashboards

4. Objectives

The main objectives of this project are:

- Build a legal document retrieval system
- Implement semantic search using embeddings
- Store vectors using ChromaDB
- Develop a Retrieval-Augmented Generation pipeline
- Integrate a Large Language Model for question answering
- Create an interactive Streamlit dashboard
- Visualize legal document analytics

5. Technology Stack

Programming Language

- Python

Libraries & Frameworks

- Streamlit
- Pandas
- NumPy
- Plotly
- LangChain
- ChromaDB
- Sentence Transformers

AI / LLM

- Ollama
- Llama 3.2

Vector Database

- ChromaDB

Version Control

- Git
- GitHub

6. System Architecture

User Query

↓

Embedding Generation

↓

Vector Search (ChromaDB)

↓

Relevant Document Retrieval

↓

Context Construction

↓

LLM (Llama 3.2 via Ollama)

↓

Generated Response

↓

Streamlit Dashboard

7. Project Workflow

Step 1: Data Understanding

The legal dataset was analyzed to understand:

- Document categories
- Legal domains
- Text structure
- Metadata

Notebook:

- 01_data_understanding.ipynb

Step 2: Data Profiling

Exploratory analysis was performed to identify:

- Missing values
- Data distribution
- Text statistics

Notebook:

- 02_data_profiling.ipynb

Step 3: Text Chunking

Large legal documents were divided into smaller chunks to improve retrieval accuracy.

Notebook:

- 03_chunking.ipynb

Step 4: Embedding Generation

Semantic embeddings were generated using Sentence Transformers.

Notebook:

- 04_embeddings.ipynb

Step 5: Vector Store Creation

Generated embeddings were stored in ChromaDB for efficient similarity search.

Notebook:

- 05_vector_store.ipynb

Step 6: Retrieval Pipeline

A retrieval system was developed to fetch the most relevant document chunks based on user queries.

Notebook:

- 06_retrieval.ipynb

Step 7: RAG Pipeline

The Retrieval-Augmented Generation pipeline combines retrieved context with Llama 3.2 to generate accurate responses.

Notebook:

- 07_rag_pipeline.ipynb

8. Dashboard Development

A Streamlit application was developed to provide an interactive user interface.

Main Dashboard Modules:

Case Insights

Displays:

- Total Documents
- Legal Cases
- Chunks Indexed
- RAG Accuracy

Smart Search

Allows users to ask legal questions using natural language.

RAG Analytics

Provides analytics and retrieval performance insights.

Knowledge Base

Displays legal domain distribution and stored document statistics.

9. Project Structure

Legal-Rag-Assistant

data/

└─ raw/

└─ processed/

└─ chroma_db/

notebooks/

└─ 01_data_understanding.ipynb

└─ 02_data_profiling.ipynb

└─ 03_chunking.ipynb

└─ 04_embeddings.ipynb

└─ 05_vector_store.ipynb

└─ 06_retrieval.ipynb

└─ 07_rag_pipeline.ipynb

src/

└─ rag/

└─ streamlit_app/

└─ utils/

README.md

requirements.txt

config.py

10. Features Implemented

- Semantic Legal Search
- Vector Database Storage
- Document Chunking
- Embedding Generation
- Retrieval-Augmented Generation
- Interactive Dashboard
- Legal Analytics Visualization
- Knowledge Base Explorer
- Streamlit User Interface
- GitHub Integration

11. Results

The developed system successfully:

- Retrieves relevant legal information using semantic similarity search
- Generates context-aware answers using LLMs
- Stores and retrieves embeddings efficiently using ChromaDB
- Visualizes legal insights through an interactive dashboard
- Demonstrates an end-to-end implementation of Retrieval-Augmented Generation

12. Challenges Faced

- Managing large legal documents
- Optimizing chunk size for retrieval
- Dependency management
- Integrating ChromaDB with LangChain
- Streamlit deployment and dashboard design

13. Future Enhancements

- Multi-user authentication
- PDF upload and processing
- Legal citation generation
- Case recommendation engine
- Cloud deployment
- Real-time legal knowledge updates
- Advanced document summarization

14. Conclusion

The Legal RAG Assistant successfully demonstrates the implementation of a Retrieval-Augmented Generation system for legal document retrieval and analysis.

The project integrates modern AI technologies such as vector databases, embeddings, semantic search, and large language models to provide intelligent legal assistance. The Streamlit dashboard offers a user-friendly interface for searching legal information and analyzing knowledge base statistics.

This project highlights the practical application of Generative AI and Retrieval-Augmented Generation in the legal domain and serves as a foundation for future enterprise-scale legal intelligence systems.

15. References

1. LangChain Documentation
2. ChromaDB Documentation
3. Streamlit Documentation
4. Ollama Documentation
5. Sentence Transformers Documentation
6. Python Official Documentation
7. Plotly Documentation
8. Llama 3.2 Documentation